

Laboratório de desenvolvimento de software

Apresentação da disciplina

Unidades e subunidades

- Unidade 1 - Planejamento de projeto de software
 - 1.1 Ambientes de desenvolvimento de software
 - 1.2 Codificação em camadas
 - 1.3 Arquitetura de software
- Unidade 2 - Interface gráfica
 - 2.1 Principais componentes gráficos
 - 2.2 Desenvolvimento de software
 - 2.3 Estudos de caso

Unidades e subunidades

- Unidade 3 - Integração com banco de dados
 - 3.1 Fundamentos de bancos de dados
 - 3.2 Desenvolvimento de software
 - 3.3 Configuração e comunicação em rede
- Unidade 4 - Desenvolvimento de software
 - 4.1 Gerenciamento de dependências
 - 4.2 Repositórios de projetos
 - 4.3 Implementação e distribuição

Planejamento

- Semana 1 – Apresentação da disciplina e revisão orientação a objetos
- Semana 2 – Revisão orientação a objetos
- Semana 3 - Ambientes de desenvolvimento de software, Codificação em camadas e Arquitetura de software
- Semana 4, 5 e 6 - Interface gráfica com swing
- Semana 7 – Produto de aprendizagem 1
- Semana 8 - Repositórios e versionamento

Planejamento

- Semana 9 e 10 – Banco de dados - conexão com mysql
- Semana 11 – Projeto
- Semana 12 e 13 - Java Persistence API
- Semana 14 - Gerenciamento de dependências
- Semana 15 – Produto de aprendizagem 2
- Semana 16 e 17 - Projeto Final

Planejamento

- Semana 18 – Apresentação do projeto final
- Semana 19 – Produto de aprendizagem 3
- Semana 20 – Finalização da disciplina

Avaliações

- Para o aluno ser aprovado precisará ter no mínimo 75% de frequência às aulas. A presença em sala de aula é importantíssima! Alunos com média semestral igual ou superior a 6 estarão aprovados sem exame.
- A Nota Final (NF) será composta da seguinte forma:
- $NF = [Nota\ 1(N1) + Nota\ 2(N2) + Nota3(N3)] / 3$, em que:

Avaliações

- Nota 1 (N1) composta por:
 - Exercícios em aula, com peso = 3,0. Os exercícios deverão ser entregues sempre 1 semana após o professor anunciá-los;
 - Produto de Aprendizagem 1, com peso = 7,0.
- Nota 2 (N2) composta por:
 - Exercícios em aula, com peso = 3,0. Os exercícios deverão ser entregues sempre 1 semana após o professor anunciá-los;
 - Produto de Aprendizagem 2, com peso = 7,0.
- Nota 3 (N3) composta por:
 - Exercícios em aula, com peso = 3,0. Os exercícios deverão ser entregues sempre 1 semana após o professor anunciá-los;
 - Produto de Aprendizagem 3, com peso = 7,0.

Laboratório de desenvolvimento de software

Revisão – Programação Orientada a objetos

Programação orientada a objetos

- Conceitos da OO:
- Em orientação a objetos, uma classe é uma estrutura que abstrai um conjunto de objetos com características similares.
- A classe define o comportamento dos objetos através de métodos e atributos
 - Métodos = funções na linguagem estruturada
 - Atributos = variáveis na linguagem estruturada

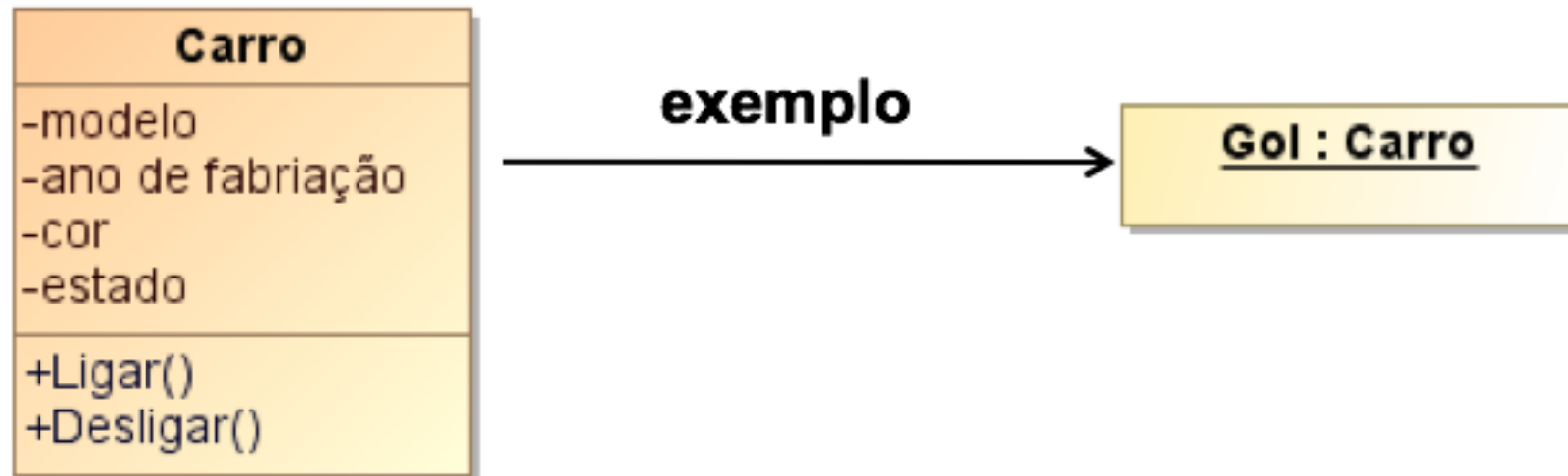
Programação orientada a objetos

- A classe descreve as características e funcionalidades dos objetos



Programação orientada a objetos

- Conceitos da OO:
- O objeto é a instância de uma classe
- Ex: Classe Carro e Objeto Gol



Programação orientada a objetos

- Conceitos da OO:
- É possível ter vários objetos a partir de uma classe

Celta : Carro

Corsa : Carro

Civic : Carro

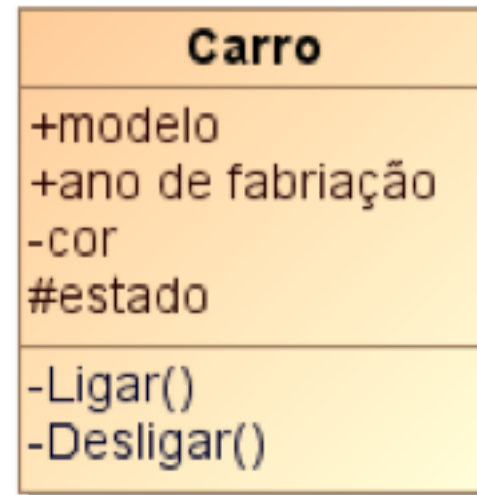
Meriva : Carro

Programação orientada a objetos

- Estado e Comportamento
 - O estado de um objeto é representado por seus atributos
 - O comportamento é representado pelos seus métodos
- Exemplo:
 - Classe Carro
 - Objeto Gol
 - Atributos: modelo, ano, cor, estado
 - Métodos: ligar, desligar

Programação orientada a objetos

- Encapsulamento:
 - É o empacotamento dos atributos e métodos numa classe
 - Proteção dos dados:
 - Público (+)
 - Privado (-)
 - Protegido (#)



Programação orientada a objetos

- Em resumo:
 - Classe: representa um conjunto de objetos
 - Subclasse: classe filha que herda os atributos e os métodos da classe mãe (Herança)
 - Objeto: instância de uma classe
 - Atributo: características do objeto
 - Método: funcionalidades do objeto

Programação orientada a objetos

- Subclasse: classe filha que herda os atributos e os métodos da classe mãe (Herança)
- Mensagem: troca de informação entre os objetos
- Associação: utilização de recursos entre objetos
- Abstração: classe não instanciável
- Polimorfismo: métodos com muitas formas: sobrecarga, sobrescrita

Programação orientada a objetos

- Modelagem Orientada a Objetos
- Linguagem UML
 - - Padrão para modelagem de sistemas
 - - Amplamente difundida e consolidada
 - - Largamente utilizada na indústria de software
- Mais informações em: <http://www.uml.org>

Programação orientada a objetos

- Perguntas e Respostas:
 - O que é uma Classe?
 - O que é um Objeto?
 - O que é um Método?
 - O que é um Atributo?

Classes e atributos

- Primeira classe em Java

```
3 public class Pessoa {  
4     public String nome;  
5     public int idade;  
6 }
```

- - Qual o nome da classe?
- - Qual o nome do arquivo .java?
- - A classe pode ter um nome diferente do nome do arquivo?
- - A classe possui atributos e métodos? Quais?
- - O que faz a classe Pessoa?
- - Ela é executável?

Classes e atributos

- Instanciando a classe Pessoa

```
3 public class Pessoa {  
4     public String nome;  
5     public int idade;  
6 }
```

Pessoa.java

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6         Pessoa p = new Pessoa();  
7         System.out.println("A pessoa foi instanciada!");  
8     }  
9 }
```

Principal.java

- - Instanciar = criar um objeto
- - Qual o objeto criado?
- - Qual o resultado da execução da classe Principal?

Classes e atributos

- Jogo rápido: exibir apenas a idade de três pessoas, ou seja: criar três objetos do tipo Pessoa;

Classes e atributos

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6         Pessoa p1 = new Pessoa();  
7         p1.idade = 20;  
8         Pessoa p2 = new Pessoa();  
9         p2.idade = 22;  
10        Pessoa p3 = new Pessoa();  
11        p3.idade = 24;  
12        System.out.println("A idade da pessoa 1 é: "+p1.idade);  
13        System.out.println("A idade da pessoa 2 é: "+p2.idade);  
14        System.out.println("A idade da pessoa 3 é: "+p3.idade);  
15    }  
16 }
```

Classes e atributos

- Jogo rápido (2): crie uma classe chamada Professor que contenha um atributo público chamado nome, do tipo String. Crie também uma classe chamada Laboratorio, que contenha um atributo público, chamado local, do tipo String.
- Além disso, crie uma classe executável chamada Disciplina, responsável por instanciar as classes Professor e Laboratorio, definindo valor aos atributos e exibindo na tela o resultado da criação desses objetos.
- Por exemplo: o resultado da execução da classe Disciplina deve ser algo similar com:
 - O nome do professor é: Ricardo da Silva
 - O local da aula é: Sala 108

Classes e atributos

```
3 public class Professor {  
4     public String nome;  
5 }
```

```
3 public class Laboratorio {  
4     public String local;  
5 }
```

```
3 public class Disciplina {  
4     public static void main(String[] args) {  
5         Professor p = new Professor();  
6         p.nome = "Ricardo da Silva";  
7  
8         Laboratorio lab = new Laboratorio();  
9         lab.local = "Sala 108";  
10  
11         System.out.println("O nome do professor é: "+p.nome);  
12         System.out.println("O local é: "+lab.local);  
13     }  
14 }
```

Exercícios

- 1 - Crie uma classe Pessoa com os atributos nome, idade e gênero. Imprima as informações da pessoa na tela.
- 2 - Crie uma classe Livro com os atributos título, autor e ano de publicação. Imprima as informações do livro na tela.

Métodos

- Primeira classe em Java com atributos e métodos

```
3 public class Carro {  
4     public String marca;  
5     public String modelo;  
6  
7     public void alugar() {  
8         System.out.println("Carro "+marca+" "+modelo+" alugado!");  
9     }  
10  
11     public void devolver() {  
12         System.out.println("Carro "+marca+" "+modelo+" devolvido!");  
13     }  
14 }
```

- Qual o nome da classe? A classe possui atributos e métodos? Quais? O que faz a classe? Ela é executável?

Métodos

```
3 public class Carro {
4     public String marca;
5     public String modelo;
6
7     public void alugar() {
8         System.out.println("Carro "+marca+" "+modelo+" alugado!");
9     }
10
11     public void devolver() {
12         System.out.println("Carro "+marca+" "+modelo+" devolvido!");
13     }
14 }
```

```
3 public class Locadora {
4
5     public static void main(String[] args) {
6         Carro c = new Carro();
7         c.alugar();
8         c.devolver();
9     }
10 }
```

Métodos

- Deu algo errado?
 - Não temos nada nos atributos...
- Jogo rápido:
 - Defina algum valor para os atributos!

Métodos

- Jogo rápido:
 - Defina algum valor para os atributos!

```
5 public class Locadora {  
6  
7     public static void main(String[] args) {  
8         Carro c = new Carro();  
9         c.marca = "VW";  
10        c.modelo = "Fusca";  
11        c.alugar();  
12        c.devolver();  
13    }  
14 }
```

Jogo rápido

- Implemente a classe LocadoraVeiculos para fazer a leitura do modelo e marca de um carro, instanciar a classe Carro.
- Crie um método na classe Carro para exibir os dados do veículo.

Jogo rápido

```
3 public class Carro {
4     public String marca;
5     public String modelo;
6
7     public void alugar() {
8         System.out.println("Carro "+marca+" "+modelo+" alugado!");
9     }
10
11     public void devolver() {
12         System.out.println("Carro "+marca+" "+modelo+" devolvido!");
13     }
14
15     public void exibirDados() {
16         System.out.println("Carro: "+marca+"\nModelo: "+modelo);
17     }
18 }
```

Jogo rápido

```
5 public class Locadora {  
6  
7     public static void main(String[] args) {  
8         Carro c = new Carro();  
9         Scanner sc = new Scanner(System.in);  
10        System.out.println("Digite a marca do carro: ");  
11        c.marca = sc.nextLine();  
12        System.out.println("Digite o modelo do carro: ");  
13        c.modelo = sc.nextLine();  
14        c.exibirDados();  
15        sc.close();  
16    }  
17 }
```

Métodos - Retornos

- Os métodos podem ou não retornar um valor.
- Se um método retorna um valor, é preciso especificar qual é o tipo desse valor na declaração do método.
- O valor é retornado usando a palavra-chave `return` seguida do valor desejado.
- Por exemplo, um método que calcula a soma de dois números `double` e retorna o resultado pode ser definido da seguinte forma:

Métodos - Retornos

```
3 public class Calculadora {
4     double a, b;
5     public double somar() {
6         double resultado = a + b;
7         return resultado;
8     }
9 }

3 public class Principal {
4
5     public static void main(String[] args) {
6         Calculadora c = new Calculadora();
7         double res;
8         c.a = 10;
9         c.b = 15;
10        res = c.somar();
11
12        System.out.println("O resultado da soma é: "+res);
13    }
14
15 }
```

Métodos - Parâmetros

- Os parâmetros são valores que são passados para o método quando ele é chamado.
- Eles são usados dentro do método para realizar a tarefa desejada.
- Na declaração do método, é preciso especificar os tipos dos parâmetros e os nomes que eles terão dentro do método.
- Por exemplo, o método de soma que recebe dois parâmetros do tipo double, chamados a e b, conforme a seguir:

Métodos - Parâmetros

```
3 public class Calculadora {
4     public double somar(double a, double b) {
5         double resultado = a + b;
6         return resultado;
7     }
8 }

3 public class Principal {
4
5     public static void main(String[] args) {
6         Calculadora c = new Calculadora();
7         double res;
8         res = c.somar(10, 15);
9
10        System.out.println("O resultado da soma é: "+res);
11    }
12
13 }
```

Classes e métodos

- Crie um método para atribuir valores para marca e modelo do carro, sendo que esses valores são recebidos pelo método através de parâmetros.
- Após, implemente a classe LocadoraVeiculos para instanciar a classe Carro, chamar o método correspondente e passar os parâmetros necessários. Além de um método para atribuir os valores, crie um método que retorna os valores atribuídos.
- O resultado da execução da classe LocadoraVeiculos deve ser:
- Dados do carro: VW Gol

Classes e métodos

```
3 public class Carro {
4     public String marca;
5     public String modelo;
6
7     public void ConfiguraDados(String marcaCarro, String modeloCarro) {
8         marca = marcaCarro;
9         modelo = modeloCarro;
10    }
11
12    public String retornaMarca() {
13        return marca;
14    }
15    public String retornaModelo() {
16        return modelo;
17    }
18 }
```


Classes e métodos

```
5 public class Locadora {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         Carro c = new Carro();
10        String ma, mo;
11        System.out.println("Digite a marca do carro: ");
12        ma = sc.nextLine();
13        System.out.println("Digite o modelo do carro: ");
14        mo = sc.nextLine();
15        c.ConfiguraDados(ma, mo);
16        System.out.println("Dados do carro:" + c.retornaMarca() + " " + c.retornaModelo());
17    }
18 }
```

Classes e métodos

- Jogo rápido: crie um classe chamada Moto com três atributos (marca, modelo e cilindradas) e dois métodos (atribuir valores e retornar valores).
- Na classe LocadoraVeiculos, crie um objeto do tipo Carro e dois objetos do tipo Moto, sendo que os objetos serão criados de acordo com a solicitação desses dados ao usuário, via linha de execução.
- Após a criação dos objetos, utilize o método para retornar valores e exiba na tela o conteúdo dos objetos criados.

Construtores

Construtores

- Um método construtor pode ser utilizado para inicializar um objeto de uma classe.
- O Java requer uma chamada de construtor para todo o objeto que é criado.
- Por padrão, o compilador fornece um construtor-padrão sem parâmetros em qualquer classe que não inclua explicitamente um construtor.
- Ao declarar uma classe, é possível fornecer seu próprio construtor a fim de especificar a inicialização personalizada para os objetos da classe.

Construtores

- A palavra-chave `new` chama o construtor da classe para realizar a inicialização.
- Ou seja, quando uma classe é instanciada, o primeiro método a ser executado é o construtor daquela classe.
- A chamada de um construtor é indicada pelo nome da classe seguido por parênteses.
- O nome do construtor deve ser igual ao nome da classe

Construtores

```
3 public class Carro {  
4     public String marca;  
5     public String modelo;  
6  
7     public void exibirDados() {  
8         System.out.println("Carro: "+marca+"\nModelo: "+modelo);  
9     }  
10 }
```

- Qual o nome da classe?
- Ela possui construtor?
- O que faz a classe?

Construtores

```
3 public class Carro {  
4     public String marca;  
5     public String modelo;  
6  
7     public Carro(String modeloCarro) {  
8         modelo = modeloCarro;  
9     }  
10  
11     public void exibirDados() {  
12         System.out.println("Carro: "+marca+"\nModelo: "+modelo);  
13     }  
14 }
```

- A classe possui quantos métodos?

Construtores

```
5 public class Locadora {  
6  
7     public static void main(String[] args) {  
8         Carro c = new Carro("Fusca");  
9         c.marca = "VW";  
10        c.exibirDados();  
11    }  
12 }
```


Construtores

```
3 public class Carro {
4     public String marca;
5     public String modelo;
6
7     public Carro(String modeloCarro, String marcaCarro) {
8         modelo = modeloCarro;
9         marca = marcaCarro;
10    }
11
12    public void exhibirDados() {
13        System.out.println("Carro: "+marca+"\nModelo: "+modelo);
14    }
15 }
```

Construtores

```
5 public class Locadora {  
6  
7     public static void main(String[] args) {  
8         Carro c = new Carro("Fusca", "VW");  
9         c.exibirDados();  
10    }  
11 }
```

Construtores

- É possível ter mais de um construtor na mesma classe?
- Com o mesmo nome? Como pode?
- Resposta: a diferenciação ocorre pelo número de parâmetros presente em cada construtor no momento da instanciação.

Construtores

```
3 public class Carro {
4     public String marca;
5     public String modelo;
6
7     public Carro(String modeloCarro) {
8         modelo = modeloCarro;
9     }
10
11     public Carro(String modeloCarro, String marcaCarro) {
12         modelo = modeloCarro;
13         marca = marcaCarro;
14     }
15
16     public void exhibirDatos() {
17         System.out.println("Carro: "+marca+"\nModelo: "+modelo);
18     }
19 }
```

Construtores

```
5 public class Locadora {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         String ma, mo;
10        System.out.println("Digite a marca do carro: ");
11        ma = sc.nextLine();
12        System.out.println("Digite o modelo do carro: ");
13        mo = sc.nextLine();
14        Carro c = new Carro(mo,ma);
15        c.exibirDados();
16
17        System.out.println("Digite a marca do carro: ");
18        ma = sc.nextLine();
19        System.out.println("Digite o modelo do carro: ");
20        mo = sc.nextLine();
21        Carro c2 = new Carro(mo);
22        c2.exibirDados();
23    }
24 }
```

Exercícios

- 1) Crie uma classe ContaCorrente que obedeça à descrição abaixo:
 - A classe possui o atributo saldo do tipo float e os métodos definirSaldoInicial, depositar e sacar.
 - O método definirSaldoInicial deve atribuir o valor passado por parâmetro ao atributo saldo
 - O método depositar, deve adicionar o valor passado por parâmetro ao atributo saldo
 - O método sacar deve reduzir o valor passado por parâmetro do saldo já existente
 - Necessário verificar a condição do valor do saldo ser insuficiente para o saque que se deseja fazer.
 - O valor de retorno deve ser true (verdadeiro) quando for possível realizar o saque e false (falso) quando não for possível (public bool sacar(float valor))

Exercícios

- Crie um objeto novaConta do tipo ContaCorrente.
- Chame o método definirSaldoInicial passando o valor 1000 como parâmetro.
- Escreva o valor do atributo saldo
- Realize um saque de 500 reais (utilize o método sacar).
- Faça um depósito de 50 reais (utilize o método depositar)
- Escreva o valor do atributo saldo na tela.
- Realize um saque de 600 reais.
- Escreva o valor do atributo saldo na tela